

Managed Runtime (MRT) Data Store

The MRT data store stores B2C Commerce settings, such as site preferences and Page Designer changes, and makes those settings available to Storefront Next. Your Storefront Next app accesses settings from the data store, which makes settings retrieval faster and more performant because there is no need for B2C Commerce API (SCAPI) calls to get those settings. Also, page descriptions for changed Page Designer pages are stored in the data store where the storefront gets them, eliminating the need to build page layouts for each shopper request.

In this section:

- [Access Site Preferences from the MRT Data Store](#)
- [Page Designer Change Deployment to Storefront Next](#)

Access Site Preferences from the MRT Data Store

Use the Managed Runtime (MRT) data store to get site preferences that administrators set in Business Manager. Access site preferences in different contexts within your Storefront Next app.

Configuration

The data store middleware is bundled and exported as `dataStoreMiddleware`. Register it in `src/root.tsx`. Make sure that `siteContextMiddleware` runs first so that the site-preferences middleware can resolve the current site ID.

Contents

[Access Site Preferences from the MRT Data Store](#)

[Local Development](#)

[Page Designer Change](#)

[Deployment to Storefront Next](#)

[See Also](#)



store middleware for you.

```
// src/root.tsx
import { createStoreMiddleware }
export const middleware: Middle
  siteContextMiddleware,
  ...dataStoreMiddleware,
  // ... rest of middleware
];
```

MRT sets these required environment variables in production:

- `AWS_REGION`
- `MOBIFY_PROPERTY_ID`
- `DEPLOY_TARGET`

Optionally, set `SFNEXT_DATA_STORE_UNAVAILABLE_MODE = fallback` to return an empty object instead of throwing when the data store is unreachable.

Usage in root.tsx (Server Middleware)

```
// src/root.tsx
import {
  getCustomGlobalPreferences,
  getSitePreferences,
} from "@salesforce/storefront-
const maintenanceMiddleware: Mi
const sitePrefs = getSitePref
const globalPrefs = getCustom
if (sitePrefs.maintenanceMode
  return new Response("Mainte
}
return next();
};
```

Usage in Route Loaders

Use the sync getters in any server loader or action. Both getters return `Record<string, unknown>`, which you cast or validate at the call site.

```
// src/
import {
  getCustomGlobalPreferences,
  getSitePreferences,
} from "@salesforce/storefront-
const maintenanceMiddleware: Mi
const sitePrefs = getSitePref
const globalPrefs = getCustom
if (sitePrefs.maintenanceMode
  return new Response("Mainte
}
return next();
};
```



```
6  
7 export async function loader({  
8   const sitePrefs = getSitePref  
9   const globalPrefs = getCustom  
10  
11   return {  
12     showBanner: Boolean(sitePre  
13     enableChat: Boolean(globalP  
14   };  
15 }
```

Usage in React Components

To access site preferences in React components, read them in the route loader and pass them to components via `useLoaderData`. There is no client-side hook for site preferences.

```
1 // src/routes/home.tsx  
2 export default function Home({  
3   const { showBanner, enableCha  
4  
5   return (  
6     <div>  
7       {showBanner && <HeroBanne  
8       {enableChat && <ChatWidge  
9     </div>  
10   );  
11 }
```

Local Development

Use `MRT_DATA_STORE_DEFAULTS` to seed values without a real data store. Prefix site preference keys with the site ID.

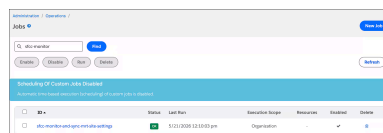
```
1 MRT_DATA_STORE_DEFAULTS='{ "cus  
2   "RefArch-custom-site-preferen
```

Data Store Keys:

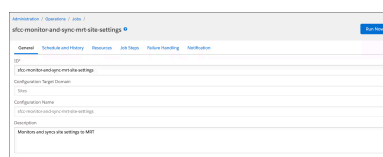
- Global: `custom-global-preferences`
- Site: `{siteId}-custom-site-preferences`



store based on the B2C target info and Site IDs configured for the Managed Runtime environment. Site preferences can take up to 5 minutes to take effect. To sync site preferences manually, run the `sfcc-monitor-and-sync-mrt-site-settings` job in Business Manager under **Administration > Operations > Jobs**.



This image shows the details of the site preferences job.



The results of the manually run job are available in **Administration > Operations > Job History**.

Page Designer Change Deployment to Storefront Next

When a merchant makes a change to Page Designer pages and components in staging, a system job deploys the changes to Managed Runtime (MRT) every 5 minutes. B2C Commerce prebuilds each Page Designer page and sends it to the MRT data store. The storefront reads the page description from the data store instead of assembling the page on every shopper request. This approach improves storefront performance by eliminating the need to build page layouts dynamically for each shopper.

You don't author manifests manually. The system automatically builds them from your Page Designer pages without any changes required from you in Business Manager.



Kit, SFRA, or Site Genesis.

How Changes in Page Designer Reach Storefront Next

Two preconfigured system jobs deliver manifests to your storefront.

- **sfcc-push-page-manifests** :
The system runs this job automatically on production after replication. This job picks up the manifests that have been replicated from staging and pushes them to the MRT data store.
- **sfcc-generate-and-push-page-manifests** : Run this job manually on production or development instances to push changes to the data store immediately. From Business Manager, go to **Administration > Operations > Jobs**, locate the job, and run it. This job generates manifests from the current instance and pushes them to the data store.

After either job runs, check the results in Business Manager at **Administration > Operations > Job History**. A successful job shows a status of **OK** or **Finished**. If the job fails, review the log details to identify the issue.

Typical Page Update Flow

Follow this workflow to publish Page Designer changes to your storefront:

1. Author content: A merchant authors page content in Page Designer on the staging instance.
2. Replicate to production: An administrator replicates the changes from staging to production.
3. Wait for automatic sync: The system automatically runs the **sfcc-push-page-manifests** job on production after replication completes.

[Try for free](#)

Update Pages Changed Directly on Production

If you make changes directly on production without replicating from staging, the automatic job doesn't pick them up. To push these changes immediately:

1. From Business Manager, go to **Administration > Operations > Jobs**.
2. Locate the `sfcc-generate-and-push-page-manifests` job.
3. Click **Run** to execute the job.
4. Verify that the job completed successfully in **Administration > Operations > Job History**.
5. Check the affected pages on your storefront to confirm the changes appear.

See Also

- [Page Designer Content Management for Merchants](#)

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblaze
Events and Webinars
Partner Connect
Blog
Salesforce Stack Overflow
Salesforce Developers



Try for free



United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#)

[Use of Cookies](#) [Cookie Preferences](#)  [Your Privacy Choices](#)